

Лабораторная работа № 1

Вычисление числа π с использованием OpenMP

Реализованы последовательная и параллельная версии суммирования ряда Лейбница на языке C; параллельный цикл использует **parallel for reduction(+:sum)** и `schedule(static)`. Параметр `n` задаёт число слагаемых ($i = 0, \dots, n - 1$); программа выводит π , ошибку относительно M_{PI} , время и фактическое число потоков, а замеры выполнены через `omp_get_wtime()`.

Условия: Apple M4, 10 ядер (4 производительных + 6 энергоэффективных), 16 ГБ; macOS 26.3.1; Apple Clang 17.0.0, LLVM libomp 23.1.2. Оптимизация -O2, стандарт C11, OpenMP включён через -Xpreprocessor -fopenmp и -lomp; fast-math не использовался.

Методика: медианы 3 отдельных запусков каждой конфигурации; порядок конфигураций перемешан. Измеряется только вычисление, включая вход в параллельную область и редукцию, без вывода. Динамическое изменение числа потоков отключено. Ускорение $S = T_{seq} / T_{omp}$.

1. Влияние числа потоков: $n = 10^9$

Последовательная версия: $T_{seq} = 0,52286$ с.

Потоки	Время OpenMP, с	Ускорение
1	0,59221	0,88
2	0,30659	1,71
4	0,15686	3,33
8	0,09866	5,30
16	0,09025	5,79

2. Влияние размера задачи: 8 потоков

n	Последовательно, с	OpenMP, с	Ускорение
10^7	0,00546	0,00127	4,29
10^8	0,05237	0,01076	4,87
10^9	0,52286	0,09866	5,30
10^{10}	5,23249	0,95714	5,47

Вывод

При увеличении числа потоков от 1 до 16 время OpenMP уменьшилось с 0,59221 до 0,09025 с, а максимальное ускорение относительно последовательной версии составило 5,79 раза. Один поток OpenMP оказался медленнее последовательной версии из-за накладных расходов и различий скомпилированных циклов. После 8 потоков рост ускорения заметно замедлился: переход к 16 потокам дал лишь 9,3% прироста. Это согласуется с ограниченным числом физических ядер и их неодинаковой производительностью. При росте n от 10^7 до 10^{10} ускорение на 8 потоках выросло с 4,29 до 5,47, а эффективность $S / 8$ - примерно с 53,6% до 68,3%. На более крупных задачах накладные расходы составляют меньшую долю времени, поэтому OpenMP используется эффективнее. Увеличение n примерно в 10 раз приводит к сопоставимому росту времени вычисления, что соответствует линейной сложности. При $n = 10^{10}$ абсолютная ошибка обеих версий во всех запусках не превысила 1.02×10^{-10} ; небольшие различия результатов связаны с порядком сложения чисел `double`.